VITE & GOURMAND

DOCUMENTATION TECHNIQUE

Évaluation en cours de formation

Graduate Développeur Angular
Titre professionnel Développeur Web et Web Mobile

Maeva Bomy

Promo JANVIER-FEVRIER 2027

Application en production :
https://vite-gourmand-ecf-ad74809e59a7.herokuapp.com/

Sommaire

1. Présentation et objectifs du projet
2. Réflexion technologique initiale
3. Configuration de l'environnement de travail
4. Architecture technique de l'application
5. Organisation du code et responsabilités
6. Modélisation des données relationnelles
7. Base de données non relationnelle
8. Règles de gestion principales
9. Diagramme d'utilisation
10. Diagramme de séquence
11. Sécurité de l'application
12. Services externes et e-mails
13. Gestion de versions avec Git
14. Déploiement de l'application
15. Vérifications et tests réalisés
16. Limites et pistes d'amélioration
17. Conclusion

1. Présentation et objectifs du projet

Vite & Gourmand est une application web développée dans le cadre de l'ECF du titre professionnel Développeur Web et Web Mobile. L'entreprise fictive est un traiteur Bordelais géré par Julie et José. L'objectif du projet est de rendre leurs menus accessibles en ligne, de permettre la commande d'une prestation et de proposer des espaces adaptés aux clients, aux employés et à l'administrateur.

Le cahier des charges impose notamment une base de données relationnelle, une base de données non relationnelle, une application déployée et fonctionnelle, ainsi qu'une documentation technique comprenant la réflexion technologique, l'environnement de travail, un modèle de données, un diagramme d'utilisation, un diagramme de séquence et la démarche de déploiement.

1.1 Périmètre fonctionnel

- Partie publique : accueil, menus, détail des menus, avis validés, horaires, contact, mentions légales et conditions générales de vente.
- Authentification : inscription, connexion, déconnexion et réinitialisation du mot de passe.
- Espace client : création et suivi des commandes, modification ou annulation selon le statut, gestion du profil et dépôt d'avis.
- Espace employé : gestion des commandes, des menus, des plats, des horaires et modération des avis.
- Espace administrateur : droits identiques à ceux des employés ainsi que la gestion des comptes employés et consultation des statistiques.
- Envoi d'e-mails : bienvenue, réinitialisation, confirmation de commande, suivi de commande, création de compte employé et formulaire de contact.

1.2 Contraintes structurantes

Contrainte	Réponse technique
Application web dynamique	PHP 8.4.x orienté objet, architecture MVC et rendu serveur.
Base relationnelle	MariaDB avec accès PDO et clés étrangères.
Base non relationnelle	MongoDB Atlas pour les statistiques de commandes par menu.
Filtres dynamiques	JavaScript natif côté navigateur.
Déploiement public	Heroku avec JawsDB Maria et MongoDB Atlas.
E-mails réels en production	API transactionnelle avec Brevo.
Recherche d'adresse et distance routière	Nominatim et OSRM via cURL.
Sécurité	CSRF, hash des mots de passe, contrôle des rôles, sessions sécurisées, variables d'environnement.

2. Réflexion technologique initiale

Le choix de la stack a été guidé par trois objectifs : respecter les contraintes de l'ECF, conserver une architecture compréhensible et maîtrisable, et limiter les dépendances afin de démontrer les mécanismes fondamentaux d'une application web sécurisée.

2.1 Front-end

Le front-end repose sur HTML5, CSS3, Bootstrap 5 et JavaScript natif. Bootstrap facilite la mise en page responsive et fournit des composants connus, tandis que le CSS personnalisé permet de conserver l'identité visuelle de Vite & Gourmand. JavaScript est utilisé uniquement lorsque l'interaction doit être dynamique, par exemple pour les filtres de menus, la recherche d'adresse ou le calcul du prix.

2.2 Back-end

Le back-end est développé en PHP 8.4.x orienté objet, sans framework. Une architecture MVC a été mise en place pour séparer l'affichage, la logique de traitement, les règles métier et l'accès aux données. Ce choix permet de montrer explicitement le fonctionnement du routage, des contrôleurs, des modèles, des services et des vues.

2.3 Données

MariaDB est utilisée pour les données nécessitant des relations fortes et des contraintes d'intégrité : utilisateurs, menus, plats, commandes, statuts, avis et messages de contact. PDO permet d'utiliser des requêtes préparées et de centraliser la connexion.

MongoDB Atlas répond à l'exigence d'une base non relationnelle. La collection menu_order_statistics contient un document par menu avec son identifiant, son titre, le nombre de commandes et une date de mise à jour. Les données sont préparées depuis MariaDB, synchronisées vers MongoDB puis relues depuis MongoDB pour l'affichage administrateur.

2.4 Déploiement et services externes

Heroku a été retenu pour héberger l'application PHP et gérer simplement les variables d'environnement. JawsDB Maria fournit la base relationnelle de production et MongoDB Atlas héberge la base NoSQL. Brevo assure l'envoi des e-mails transactionnels. Nominatim et OSRM fournissent respectivement la recherche d'adresses et le calcul de distance routière.

Technologie / service	Justification	Justification
PHP 8.4.x	Back-end	Choisi pour appliquer directement les notions vues en formation et mieux comprendre le fonctionnement d'une architecture MVC sans dépendre d'un framework.
Bootstrap 5	Interface	Responsive rapide et composants standards.
JavaScript natif	Interactions	Suffisant pour les filtres, recherches et calculs dynamiques.
MariaDB	Données relationnelles	Gestion des relations, contraintes et transactions.
MongoDB Atlas	NoSQL	Répond à l'exigence NoSQL et stocke les statistiques par menu.
DBeaver	Administration / visualisation de la base relationnelle.	Utilisé pour administrer et visualiser la base de données relationnelle. Son interface graphique facilite la consultation des tables, relations et données.

Docker / Docker Compose	Environnement local reproductible	Permet de reproduire facilement l'environnement de l'application avec PHP, Apache, MariaDB et phpMyAdmin sans dépendre de la configuration locale de la machine.
XAMPP	Environnement local	Choisi pour disposer simplement sous macOS d'Apache, PHP et MariaDB dans un même environnement de développement.
Heroku	Hébergement	Choisi pour sa simplicité de déploiement, son intégration avec GitHub et parce qu'il était recommandé dans le cadre de la formation. Le GitHub Student Developer Pack permet également de bénéficier de crédits, adaptés à un projet étudiant.
Brevo	E-mails	Envoi transactionnel réel sans exposer de données confidentielles dans le dépôt.

3. Configuration de l'environnement de travail

3.1 Environnement local

Le projet a initialement été développé sous macOS avec XAMPP. Un environnement Docker a ensuite été mis en place afin de disposer d'une configuration locale reproductible et indépendante de l'installation XAMPP du poste.

/Applications/XAMPP/xamppfiles/htdocs/VITEGOURMAND

- Docker / Docker Compose : environnement local reproductible avec PHP 8.4, Apache, MariaDB et phpMyAdmin.
- XAMPP : environnement de déploiement secondaire.
- phpMyAdmin : création et contrôle de la base locale.
- DBeaver : consultation et contrôle de la base MariaDB de production.
- Git et GitHub : version et dépôt public.
- MongoDB Atlas : base NoSQL distante.
- Navigateur et outils de développement : tests fonctionnels, responsive et accessibilité.

3.2 Prérequis techniques

- Docker Desktop et Docker Compose pour l'environnement local recommandé.
- PHP 8.4.x avec les extensions PDO, pdo_mysql, MongoDB, mbstring et cURL pour une installation manuelle hors Docker.
- Apache.
- MariaDB ou MySQL.
- Accès Internet pour MongoDB Atlas, Brevo, Nominatim et OSRM.

3.3 Installation locale

1. Cloner le dépôt GitHub.

2. Copier le fichier .env.docker.example en .env.docker.

3. Renseigner dans .env.docker les variables nécessaires à MariaDB, MongoDB Atlas, Brevo et à l'application.

4. Construire et démarrer les services avec Docker Compose.

5. MariaDB initialise automatiquement le schéma et les données de démonstration à partir de database/schema.sql et database/seed.sql.

6. Le service import-images exécute automatiquement database/import-dish-images.php afin d'intégrer les images des plats dans MariaDB.

7. Ouvrir l'application sur http://localhost:8080.

8. Ouvrir phpMyAdmin sur http://localhost:8081 si nécessaire.

XAMPP reste utilisable comme environnement alternatif, mais Docker constitue désormais la méthode recommandée pour reproduire l'environnement du projet.

3.4 Dépendances déclarées pour la production

Le fichier composer.json déclare PHP 8.4.x ainsi que les extensions nécessaires au fonctionnement de l'application. La contrainte ~8.4.0 permet de rester sur la branche PHP 8.4. Le fichier composer.lock accompagne cette configuration afin de conserver une configuration reproductible entre les environnements.

```
"require": {
    "php": "~8.4.0",
    "ext-pdo": "*",
    "ext-pdo_mysql": "*",
    "ext-mongodb": "*",
    "ext-mbstring": "*"
}
```

4. Architecture technique de l'application

L'application suit une architecture MVC complétée par une couche de services. Le point d'entrée public/index.php initialise la session, définit les constantes principales, charge les classes et transmet la requête au routeur.

L'environnement local est conteneurisé avec Docker Compose. Il repose sur plusieurs services : app pour PHP 8.4 et Apache, db pour MariaDB, phpmyadmin pour l'administration de la base et import-images pour l'import automatique des images des plats dans MariaDB. Ces services communiquent sur le réseau interne créé par Docker Compose.

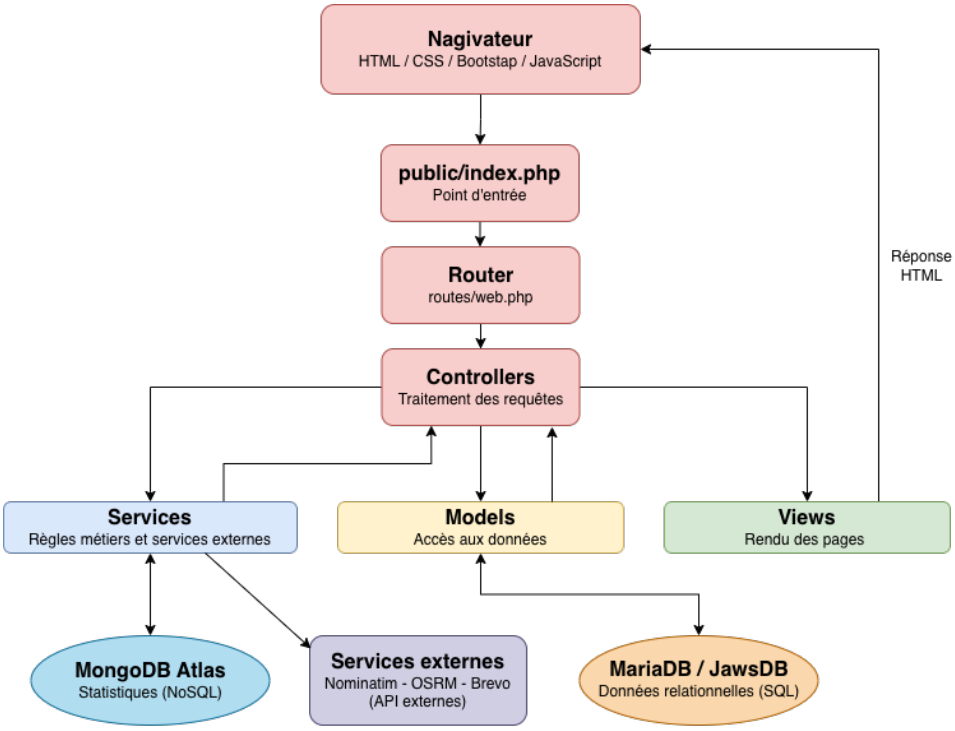


Figure 1 - Architecture générale de Vite & Gourmand

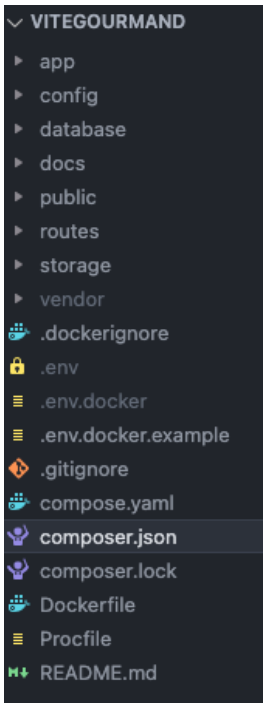
4.1 Cycle d'une requête

1. Le navigateur appelle une URL de l'application.
2. public/index.php initialise l'environnement et le routeur.
3. routes/web.php associe l'URL et la méthode HTTP à un contrôleur.
4. Le contrôleur vérifie les droits et les données reçues.
5. Les modèles interrogent MariaDB ou les services exécutent une règle métier ou un appel externe.
6. Le contrôleur prépare les données puis charge une vue.
7. La vue génère la réponse HTML renvoyée au navigateur.

4.2 Services principaux

Service	Responsabilité
Database	Connexion PDO unique à MariaDB.
MongoDatabase	Connexion au cluster MongoDB Atlas.
MongoOrderStatistics	Synchronisation et lecture des statistiques NoSQL.
OrderService	Validation des dates, créneaux et calculs de prix.
DeliveryService	Recherche d'adresse, distance routière et frais de livraison.
MailService	Construction et envoi des e-mails via Brevo en production.
Router	Association des routes aux contrôleurs.

5. Organisation du code et responsabilités



Dossier	Rôle
app/Controllers	Traitement des requêtes, contrôles d'accès et préparation des vues.
app/Models	Requêtes SQL et récupération des données.
app/Services	Logique métier partagée, routage et intégrations externes.
app/Views	Pages et composants affichés aux utilisateurs.
config	Paramètres de connexion MariaDB et MongoDB.
database	Schéma SQL, données de démonstration et scripts d'import.
public	Point d'entrée, CSS, JavaScript et images publiques.
routes	Déclaration des routes HTTP.
storage	Cache local et journaux générés par l'application.
Dockerfile	Décrit l'image Docker de l'application avec PHP 8.4 et Apache
compose.yaml	Déclare et orchestre les services Docker de l'environnement local.
.env.docker.example	Fichier d'exemple des variables d'environnement nécessaires à Docker, sans données sensibles.
docs	Livrables et documentation finale du projet.

5.1 Contrôleurs

Les contrôleurs sont séparés par domaine fonctionnel : AuthController pour l'authentification, MenuController pour le catalogue, OrderController pour la création de commande, UserController pour l'espace client, EmployeeController pour l'espace professionnel, AdminController pour les fonctions administrateur, ContactController pour le formulaire de contact et LegalController pour les pages réglementaires.

5.2 Vues

Les vues sont organisées par espace : public, authentification, commandes, client, employé et administrateur. Un layout commun centralise l'en-tête, le pied de page et le contenu principal.

6. Modélisation des données relationnelles

La base relationnelle MariaDB contient seize tables. Les relations sont protégées par des clés étrangères et les tables d'association menu_dish et dish_allergen représentent les relations plusieurs-à-plusieurs.

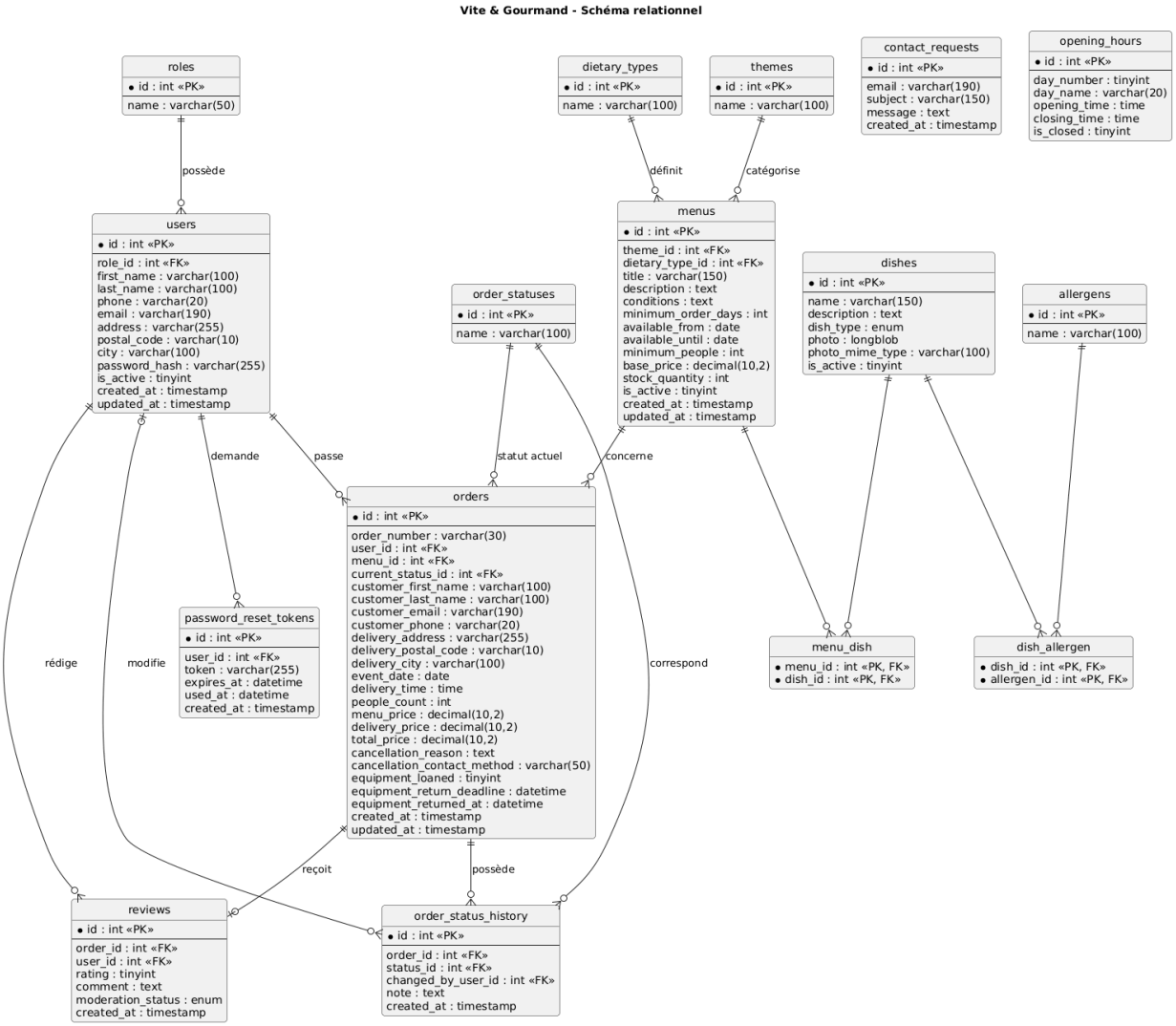


Figure 2 - Modèle conceptuel de la base relationnelle

6.1 Description des entités

Table	Rôle principal
roles	Rôles disponibles : utilisateur, employé, administrateur.
users	Comptes et informations personnelles.
password_reset_tokens	Jetons temporaires de réinitialisation du mot de passe.
opening_hours	Horaires affichés dans le site.
themes	Thèmes des menus.
dietary_types	Régimes alimentaires.
menus	Informations commerciales, conditions, prix, stock et disponibilité.
dishes	Plats, type de plat et image stockée en BLOB.
allergens	Référentiel des allergènes.
menu_dish	Association entre menus et plats.
dish_allergen	Association entre plats et allergènes.
order_statuses	Référentiel des statuts d'une commande.
orders	Commandes clients, livraison, prix, matériel et annulation.
order_status_history	Historique daté des changements de statut.
reviews	Avis clients avec note et statut de modération.
contact_requests	Messages envoyés depuis le formulaire de contact.

6.2 Intégrité et choix de conception

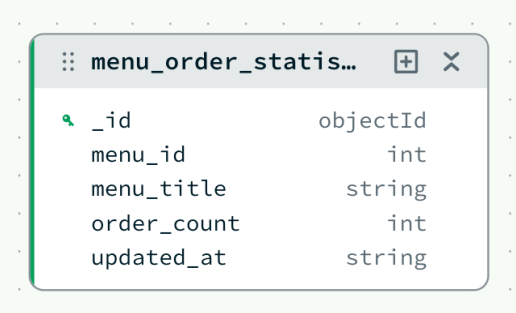
- Les adresses e-mail des utilisateurs sont uniques.
- Les clés étrangères protègent les relations entre utilisateurs, commandes, menus et statuts.
- Les relations plusieurs-à-plusieurs sont isolées dans des tables d'association.
- Une commande conserve une copie des coordonnées du client et des prix au moment de la commande afin de préserver l'historique.
- L'historique des statuts est séparé de la commande afin de retracer chaque évolution.
- Un seul avis peut être associé à une commande grâce à la contrainte UNIQUE sur reviews.order_id.

7. Base de données non relationnelle

La base MongoDB est utilisée pour les statistiques de nombre de commandes par menu, conformément à l'exigence d'utiliser une base non relationnelle.

7.1 Collection

La base de données non relationnelle MongoDB contient une collection nommée menu_order_statistics. Elle permet de stocker les statistiques de commandes associées aux différents menus.



7.2 Flux de synchronisation

1. MariaDB calcule le nombre de commandes pour chaque menu.
2. Ces statistiques sont enregistrées ou mises à jour dans MongoDB Atlas.
3. L'espace administrateur récupère ensuite ces statistiques depuis MongoDB.
4. Le chiffre d'affaires reste calculé depuis MariaDB.

   Cette organisation permet d'utiliser MariaDB pour les données principales de l'application et MongoDB pour les statistiques.

8. Règles de gestion principales

Règle	Implémentation
Mot de passe	Au moins 10 caractères, avec majuscule, minuscule, chiffre et caractère spécial.
Commande	Réservée à un compte client authentifié.
Nombre de personnes	Ne peut pas être inférieur au minimum défini par le menu.
Réduction	10 % lorsque le nombre de personnes atteint au moins minimum + 5.
Livraison Bordeaux	Gratuite.
Livraison hors Bordeaux	5 € fixes + 0,59 € par kilomètre.
Disponibilité	Respect du délai minimum et de la période éventuelle du menu.
Stock	Décrémenté à la commande, recalculé lors d'une modification et restitué lors d'une annulation.
Annulation client	Possible tant que la commande n'a pas été acceptée.
Annulation employé	Nécessite le moyen de contact et le motif.
Avis	Possible après une commande terminée puis soumis à modération.
Matériel	Suivi du prêt et du retour ; délai de 10 jours ouvrés et pénalité prévue par les CGV.

8.1 Statuts de commande

Le suivi de commande s'appuie sur des statuts enregistrés en base et sur la table order_status_history. Les principaux états prévus sont : En attente, Acceptée, En préparation, En cours de livraison, Livrée, En attente du retour de matériel, Terminée et Annulée.

9. Diagramme d'utilisation

Le diagramme présente les principaux acteurs et leurs interactions avec l'application. Les fonctionnalités publiques restent accessibles à tous les utilisateurs. L'administrateur dispose de ses fonctions propres et peut également accéder aux fonctionnalités de l'espace employé.

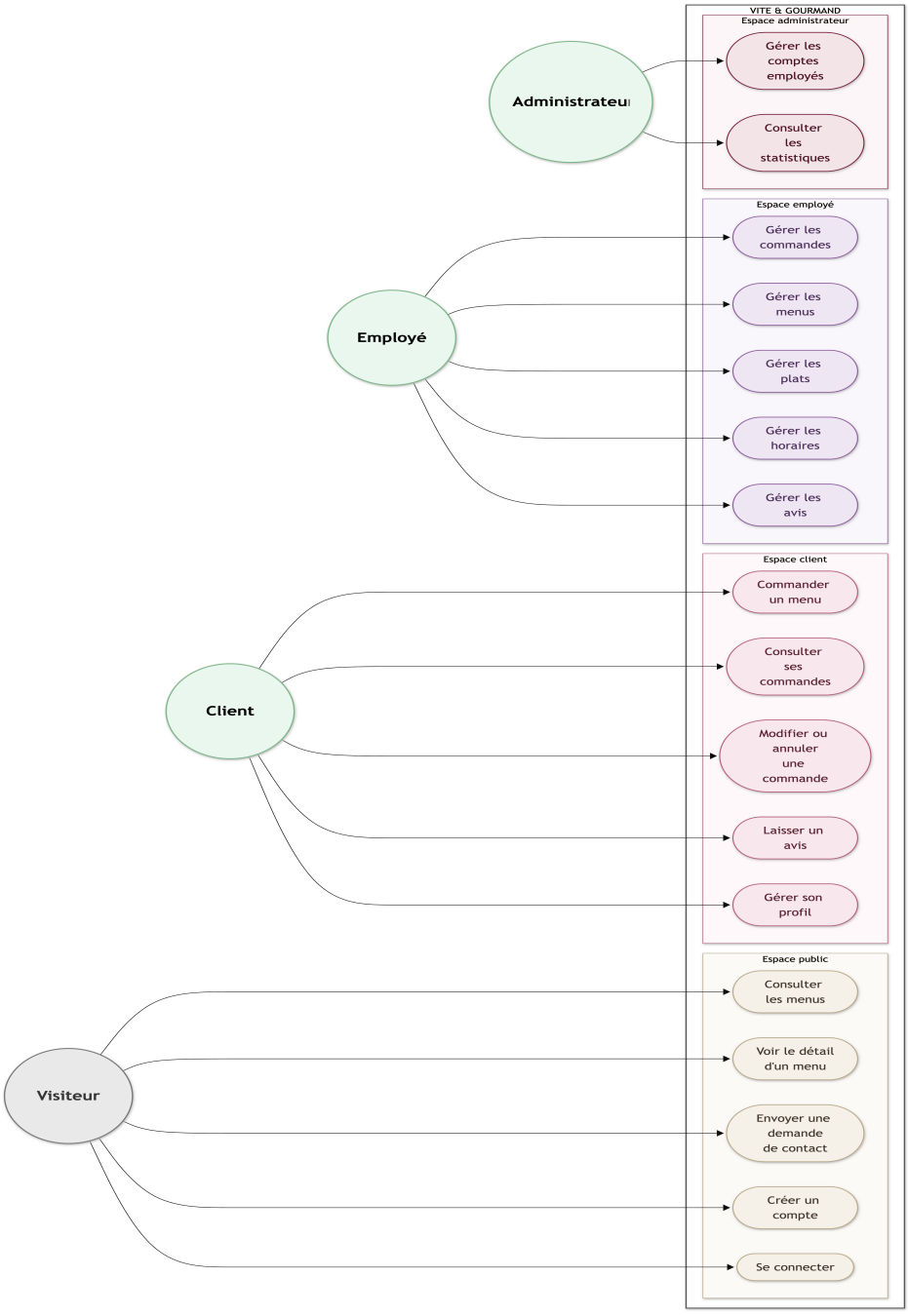


Figure 3 - Diagramme d'utilisation

10. Diagramme de séquence

Le scénario retenu est la création d'une commande, car il mobilise plusieurs couches de l'application : interface, contrôleur, services métier, base relationnelle et service d'e-mail.

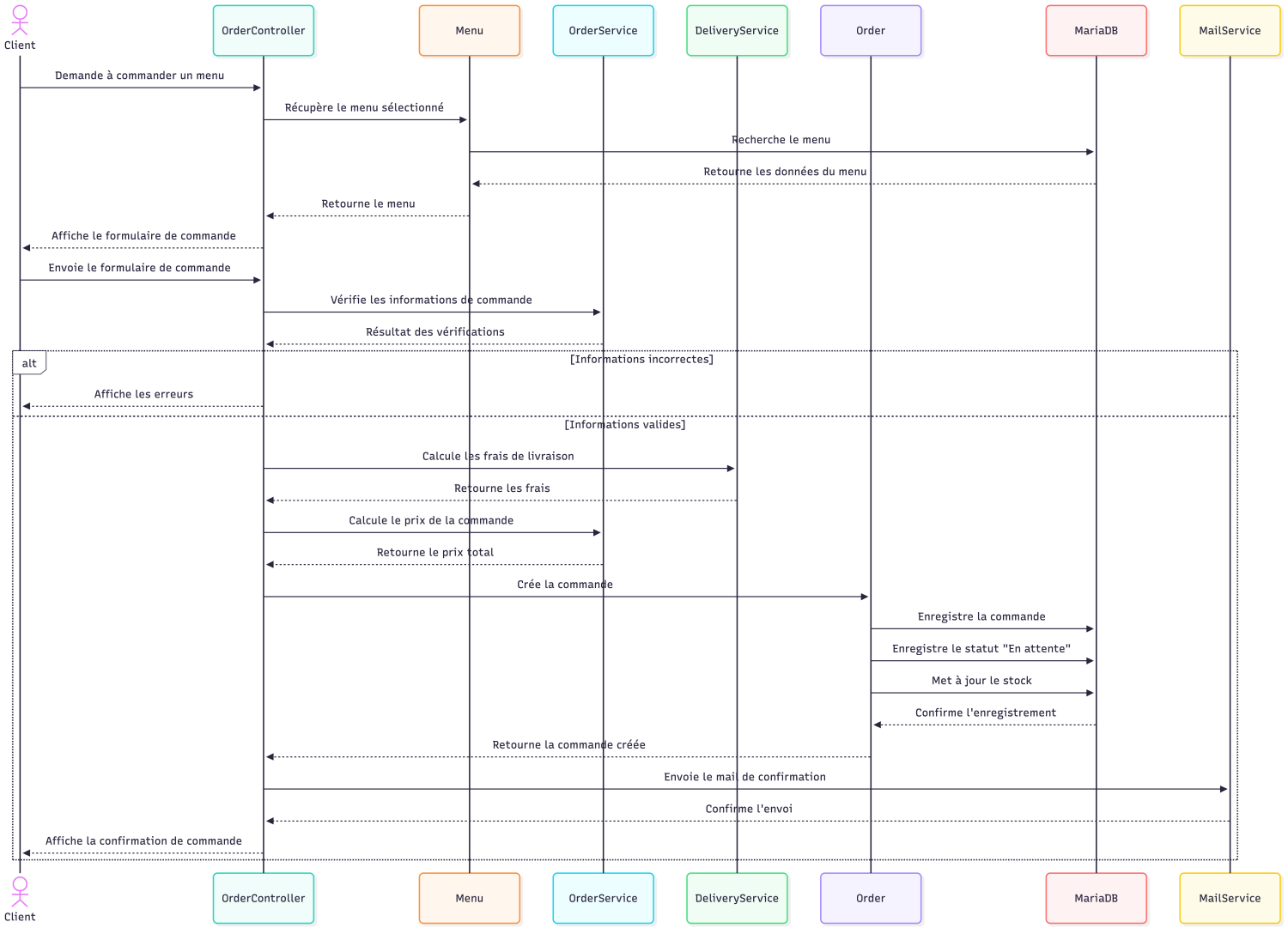


Figure 4 - Séquence simplifiée de création d'une commande

10.1 Description du scénario

1. Le client sélectionne un menu et accède au formulaire de commande.
2. OrderController récupère les informations du menu et reçoit les données saisies par le client.
3. Le contrôleur vérifie les informations de la commande et utilise OrderService pour appliquer les règles concernant notamment la date et le créneau de livraison. En cas d'erreur, les informations à corriger sont affichées au client.
4. Si les informations sont valides, DeliveryService calcule les frais de livraison et OrderService calcule le prix de la commande.

5. Order enregistre la commande dans MariaDB, crée son statut initial **« En attente »** et met à jour le stock du menu.
6. MailService tente d'envoyer l'e-mail de confirmation. Un éventuel échec de l'e-mail n'annule pas la commande déjà enregistrée.
7. Le client est ensuite redirigé vers la page de confirmation.

11. Sécurité de l'application

Les mesures de sécurité sont réparties entre le point d'entrée, les contrôleurs, les modèles et les services.

11.1 Authentification et mots de passe

- Les mots de passe sont stockés avec password_hash() et contrôlés avec password_verify().
- Les mots de passe ne sont jamais envoyés par e-mail lors de la création d'un compte employé.
- Les jetons de réinitialisation sont aléatoires, temporaires, hachés avant stockage et marqués comme utilisés après modification du mot de passe.
- La réponse du formulaire de mot de passe oublié ne révèle pas si l'adresse existe.

11.2 Sessions

- session.use_strict_mode est activé.
- L'identifiant de session est transmis uniquement par cookie.
- L'identifiant de session est régénéré après authentification.
- Les cookies utilisent HttpOnly et SameSite=Lax.
- L'attribut Secure est activé lorsque l'application détecte HTTPS, y compris derrière le proxy Heroku grâce à HTTP_X_FORWARDED_PROTO.

11.3 Formulaires et autorisations

- Des jetons CSRF sont utilisés pour les formulaires sensibles.
- Les rôles sont contrôlés côté serveur avant l'accès aux espaces client, employé et administrateur.
- Les actions administrateur sont réservées explicitement au rôle admin.
- Les données affichées sont échappées avec htmlspecialchars() lorsque nécessaire.
- Les entrées sont validées côté serveur, même lorsqu'un contrôle JavaScript existe côté client.

11.4 Accès aux données et fichiers

- Les requêtes SQL dynamiques utilisent PDO et des paramètres préparés.
- Les erreurs de connexion à la base ne révèlent pas les identifiants techniques.
- Les images téléversées sont limitées en taille et vérifiées par type MIME avant enregistrement.
- Le sujet du formulaire de contact est nettoyé des retours à la ligne afin de limiter les risques d'injection d'en-tête.

11.5 Gestion des secrets

Les secrets de production sont stockés dans les Config Vars Heroku et ne sont pas présents dans le dépôt GitHub. Pour l'environnement Docker local, les valeurs sensibles sont stockées dans le fichier .env.docker, ignoré par Git. Le fichier .env.docker.example fournit uniquement des valeurs fictives servant de modèle de configuration. Les fichiers .env, .env.docker, les journaux locaux et les fichiers de cache sont exclus du versionnement.

Les ports HTTP de l'application et de phpMyAdmin sont liés à l'interface locale 127.0.0.1 afin de ne pas les exposer directement sur le réseau de la machine.

12. Services externes et e-mails

12.1 Recherche d'adresse

DeliveryService interroge Nominatim afin de proposer des adresses françaises. Les résultats sont mis en cache localement pendant 24 heures afin de limiter les appels répétés.

12.2 Calcul de distance

OSRM calcule une distance routière entre les coordonnées de référence du traiteur à Bordeaux et l'adresse choisie. Le serveur recalcule cette distance lors de l'enregistrement de la commande afin de ne pas faire confiance à une valeur fournie uniquement par le navigateur.

12.3 E-mails transactionnels

MailService utilise l'API Brevo lorsque les variables BREVO_API_KEY et BREVO_SENDER_EMAIL sont présentes, aussi bien en production que dans l'environnement Docker local. Si ces variables ne sont pas définies, le service conserve un fonctionnement de secours basé sur la fonction PHP mail() et peut enregistrer une copie dans storage/logs/emails.log.

- Mail de bienvenue lors de l'inscription.
- Lien de réinitialisation du mot de passe.
- Confirmation d'une commande.
- Notification lorsqu'une commande est acceptée.
- Notification liée au retour du matériel.
- Invitation à déposer un avis après une commande terminée.
- Confirmation d'annulation.
- Notification lors de la création d'un compte employé.
- Transmission des demandes du formulaire de contact.

Les liens contenus dans les e-mails sont transformés en URL absolue grâce à APP_URL afin qu'ils fonctionnent depuis un client de messagerie.

12.4 Variables d'environnement utilisées

Variables	Rôles
DB_NAME	Nom de la base MariaDB utilisée dans l'environnement Docker.
DB_USER	Utilisateur MariaDB utilisé dans l'environnement Docker.
DB_PASSWORD	Mot de passe de l'utilisateur MariaDB utilisé dans l'environnement Docker.
DB_ROOT_PASSWORD	Mot de passe de l'utilisateur MariaDB utilisé uniquement pour l'initialisation du conteneur local.
JAWSDB_MARIA_URL	Connexion à MariaDB en production
MONGODB_URI	Connexion à MongoDB Atlas
MONGODB_DATABASE	Nom de la base MongoDB
MONGODB_ALLOW_INVALID_CERTIFICATES	Autorise exceptionnellement le contournement de la validation du certificat TLS en environnement local
MONGODB_TLS_CA_FILE	Chemin vers un certificat CA utilisé pour la connexion MongoDB lorsque nécessaire en local
BREVO_API_KEY	Authentification auprès de l'API Brevo
BREVO_SENDER_EMAIL	Adresse d'expédition des emails
CONTACT_RECIPIENT_EMAIL	Adresse recevant les demandes de contact
APP_URL	URL publique de l'application

Aucune valeur réelle confidentielle n'est reproduite dans cette documentation ; seules les variables utilisées par l'application et leur rôle sont présentés.

Les paramètres permettant de contourner la validation TLS sont réservés au développement local et ne doivent pas être activés en production.

13. Gestion de versions avec Git

Le dépôt public est organisé autour de trois niveaux de branches. Cette organisation correspond au flux attendu dans l'ECF.

Feature/ … -> develop   ->main

Branche	Utilisation
feature/...	Développement isolé d'une fonctionnalité ou d'un bloc technique.
develop	Intégration des fonctionnalités terminées et vérifiées.
main	Version stable destinée à la production.

Les modifications importantes sont regroupées en commits cohérents. Après vérification, une branche feature est fusionnée dans develop, puis develop est fusionnée dans main lorsque l'ensemble est stable et fonctionnel.

Dépôt GitHub public : https://github.com/Maevabomy/VITEGOURMAND

14. Déploiement de l'application

L'application est déployée sur Heroku et accessible publiquement à l'adresse suivante :

https://vite-gourmand-ecf-ad74809e59a7.herokuapp.com/

L'application est actuellement exécutée sur la stack Heroku-24 avec PHP 8.4.26.
Cette version a été vérifiée directement sur l'environnement de production après le déploiement final.

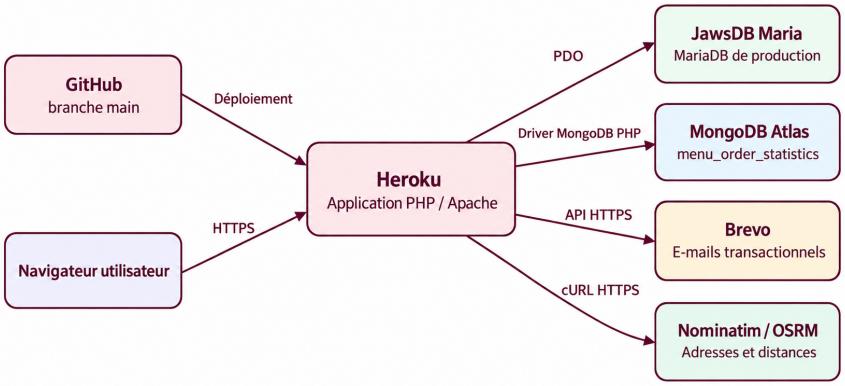


Figure 5 - Architecture de production

14.1 Préparation de l'application

• Ajout d'un Procfile à la racine du projet.

• Déclaration de PHP 8.4.x et des extensions nécessaires dans composer.json.

• Utilisation de la contrainte ~8.4.0 afin de conserver la branche PHP 8.4.

• Conservation de composer.lock.

• Adaptation de la connexion MariaDB afin de lire JAWSDB_MARIA_URL en production et les variables DB_* dans l'environnement Docker local.

• Configuration de MongoDB Atlas par variables d'environnement.

• Prise en charge du protocole HTTPS derrière le proxy Heroku pour sécuriser le cookie de session.

• Optimisation des images utilisées par l'application avant le déploiement.

14.2 Procfile

web: heroku-php-apache2 public/

Cette instruction indique à Heroku d'utiliser Apache avec le dossier public comme racine web. Les autres dossiers de l'application ne sont donc pas servis directement.

14.3 Base relationnelle de production

La base MariaDB de production est fournie par JawsDB Maria. La chaîne de connexion est stockée dans JAWSDB_MARIA_URL et analysée par config/database.php. Le schéma SQL et les données nécessaires ont été importés dans la base de production.

14.4 MongoDB Atlas

MongoDB Atlas héberge la collection menu_order_statistics. Les informations de connexion sont fournies à Heroku via MONGODB_URI et MONGODB_DATABASE. Le paramètre autorisant les certificats invalides reste désactivé en production.

14.5 Brevo

Une clé API Brevo et un expéditeur vérifié sont configurés dans Heroku. Les adresses et la clé sont lues uniquement depuis les variables d'environnement.

14.6 Déploiement Git

1. Développer et vérifier la fonctionnalité sur une branche feature.

2. Fusionner la branche dans develop après validation.

3. Fusionner develop dans main lorsque la version est stable.

4. Déployer la branche main sur Heroku avec la commande git push heroku main.

5. Vérifier le bon démarrage de la nouvelle version et effectuer un test rapide de l’application publique après déploiement.

14.7 Variables Heroku

Les Config Vars de production regroupent les chaînes de connexion, les paramètres MongoDB, la clé Brevo, l'adresse d'expédition, l'adresse de réception du formulaire de contact et l'URL publique. Les valeurs ne sont pas commitées.

14.8 Vérifications post-déploiement

- Ouverture de la page d'accueil et du catalogue.
- Connexion et déconnexion d'un client.
- Recherche et sélection d'une adresse.
- Création complète d'une commande.
- Consultation de la commande dans l'espace client.
- Mise à jour du statut depuis l'espace employé.
- Affichage des statistiques MariaDB et MongoDB dans l'espace administrateur.
- Réception réelle du mail de bienvenue.
- Réception et utilisation du mail de réinitialisation.
- Réception d'une demande provenant du formulaire de contact.
- Vérification de la version PHP réellement utilisée sur Heroku avec la commande php -v : PHP 8.4.26.

15. Vérifications et tests réalisés

15.1 Contrôles techniques

• Contrôles de syntaxe PHP avec php -l.

• Contrôle des espaces et erreurs de diff avec git diff --check via le terminal.

• Validation de la configuration Docker Compose.

• Vérification du fonctionnement local avec Docker.

• Test d’une installation propre à partir d’un clone neuf du dépôt GitHub.

• Vérification de la création automatique de la base MariaDB à partir de schema.sql et seed.sql.

• Vérification de l’import automatique des images des plats via le service import-images.

• Vérification de la connexion à MongoDB Atlas depuis l’environnement Docker.

• Vérification de l’envoi réel d’un e-mail via Brevo depuis l’environnement Docker.

• Vérification du fonctionnement de la version Heroku.

• Vérification de la version PHP utilisée en production.

• Contrôle de la base de production avec DBeaver.

• Tests des différents parcours selon les trois rôles principaux.

15.2 Tests fonctionnels principaux

Parcours	Éléments vérifiés
Public	Accueil, menus, détail, images, filtres, pages légales et contact.
Authentification	Inscription, connexion, déconnexion, mot de passe oublié.
Commande	Adresse, distance, prix, réduction, livraison, stock et confirmation.
Client	Historique, détail, modification/annulation selon statut, profil et avis.
Employé	Recherche, filtres, changement de statut, menus, plats, horaires et avis.
Administrateur	Comptes employés et statistiques MariaDB/MongoDB.
E-mails	Bienvenue, réinitialisation et formulaire de contact testés en environnement Docker et en production.

15.3 Accessibilité

Des contrôles ont été réalisés avec Lighthouse sur les pages principales, en desktop et mobile. Après correction des anomalies détectées, les pages principales testées ont obtenu 100/100 en accessibilité dans Lighthouse.

- Navigation au clavier et ordre de tabulation.
- Visibilité du focus.
- Association des champs et libellés.
- Messages d'erreur compréhensibles.
- Contrastes.
- Zoom navigateur à 200 %.
- Responsive mobile.
- Utilisation pertinente des attributs ARIA.

Ces vérifications constituent des contrôles d'accessibilité et des corrections ciblées. Elles ne constituent pas un audit complet ni une certification de conformité au RGAA.

16. Limites et pistes d'amélioration

Le projet répond au périmètre pédagogique attendu, mais plusieurs améliorations seraient pertinentes dans le cadre d'une mise en production commerciale à plus grande échelle.

- Ajouter des tests automatisés sur les règles les plus importantes, notamment le calcul des prix, les dates de commande, les changements de statut et la gestion du stock. Les tests réalisés actuellement sont principalement manuels.
- Limiter les tentatives répétées de connexion et de réinitialisation de mot de passe, afin de mieux protéger ces formulaires contre les essais automatisés.
- Améliorer la création des comptes employés en envoyant un lien temporaire permettant à l'employé de définir lui-même son mot de passe, plutôt que de faire choisir le mot de passe par l'administrateur.
- Automatiser certains contrôles avec GitHub Actions, par exemple la vérification de la syntaxe PHP avant la fusion d'une branche dans develop ou main.
- Améliorer la gestion des erreurs des services externes, afin de conserver des informations plus précises lorsqu'un appel à MongoDB, Brevo, Nominatim ou OSRM échoue et de faciliter le diagnostic.
- Développer la partie statistiques avec des fonctionnalités plus précises (nombre de commandes en attente, annulées, par période, etc.).
- Possibilité d’exporter les données au format Excel afin de faciliter la gestion.

17. Conclusion

Le projet Vite & Gourmand m'a permis de mettre en pratique les principales compétences abordées pendant la formation : structurer une application PHP avec une architecture MVC, concevoir et exploiter une base de données relationnelle, utiliser une base MongoDB, sécuriser les accès et les formulaires, intégrer des services externes, conteneuriser l'environnement local avec Docker et déployer une application accessible en ligne.

Ce projet a également été important pour moi car il m’a permis de gagner en autonomie. J’ai dû chercher des solutions, corriger des erreurs, comprendre le fonctionnement de plusieurs technologies et faire des choix techniques adaptés à mon niveau. Certaines étapes ont demandé plusieurs essais, notamment pour la gestion des bases de données, le déploiement et les services externes, mais cela m’a permis de mieux comprendre la logique globale d’une application web et de prendre davantage confiance dans ma manière de développer.

La version finale répond aux principales fonctionnalités du cahier des charges avec des espaces distincts pour les clients, les employés et l'administrateur. Le projet reste perfectible, notamment concernant les tests automatisés et certaines protections complémentaires, mais sa structure permet de poursuivre ces améliorations sans remettre en cause l'organisation générale de l'application.